NAME: C. PRIYANKA

REG NO: 192511074

7) Breadth First Search (BFS)

Aim

To develop a Python program to implement the Breadth First Search (BFS) algorithm for graph traversal.

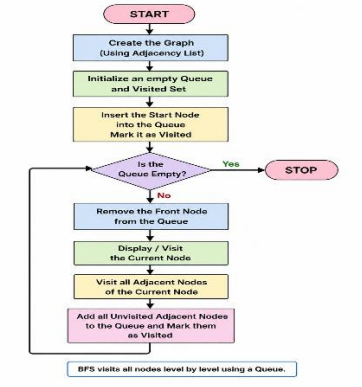
Objectives

- To understand the Breadth First Search (BFS) algorithm.
- To traverse all vertices of a graph level by level.
- To visit each node exactly once.
- To implement BFS using a queue in Python.

Algorithm

1. Start the program.
2. Create the graph using an adjacency list.
3. Initialize an empty queue and a visited list.
4. Insert the starting node into the queue.
5. Remove a node from the queue and display it.
6. Visit all unvisited adjacent nodes and add them to the queue.
7. Repeat until the queue becomes empty.
8. Stop the program.

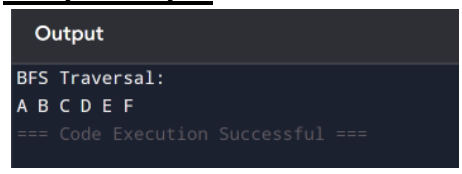
Flow Chart



Python Program

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = set()
queue = deque()

start = 'A'
visited.add(start)
queue.append(start)

print("BFS Traversal:")

while queue:
    node = queue.popleft()
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)
```

Sample Output



Result

Thus, the Python program to implement the Breadth First Search (BFS) algorithm was executed successfully, and the graph was traversed in breadth-first order.

Conclusion

The Breadth First Search (BFS) algorithm visits all nodes level by level using a queue. It is widely used in graph traversal, shortest path problems, and Artificial Intelligence applications.

8) Depth First Search (DFS)

Aim

To develop a Python program to implement the Depth First Search (DFS) algorithm for graph traversal.

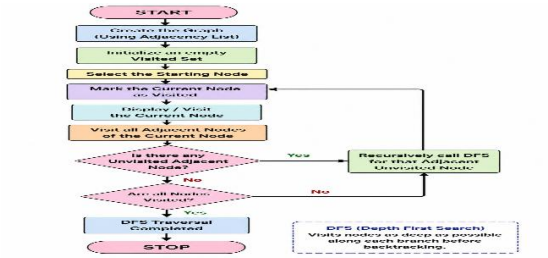
Objectives

- To understand the Depth First Search (DFS) algorithm.
- To traverse all vertices of a graph using recursion.
- To visit each node exactly once.
- To implement DFS using Python.

Algorithm

1. Start the program.
2. Create the graph using an adjacency list.
3. Initialize an empty visited set.
4. Select the starting node.
5. Mark the current node as visited.
6. Display the current node.
7. Visit each unvisited adjacent node recursively.
8. Repeat until all nodes are visited.
9. Stop the program.

Flow Chart

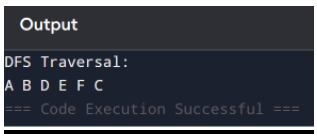


Python Program

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = set()

def dfs(node):
    if node not in visited:
        print(node, end=" ")
```

```
        visited.add(node)
        for neighbor in graph[node]:
            dfs(neighbor)

print("DFS Traversal:")
dfs('A')
```

Sample Output



Result

Thus, the Python program to implement the Depth First Search (DFS) algorithm was executed successfully, and the graph was traversed in depth-first order.

Conclusion

The Depth First Search (DFS**)** algorithm explores each branch of the graph as deeply as possible before backtracking. It is widely used in Artificial Intelligence, graph traversal, path finding, and tree traversal.